

INSTITUTO TECNOLÓGICO DE SAN JUAN DEL RÍO QUERÉTARO



Temas de Ciberseguridad

R003: Reconstrucción del código fuente con Ghidra

Anibal Atzin Zarate Hernandez 22590362

Maria Fernanda Barrera Perez 22590318

Michelle Granados Vaquero 22590329

Introducción

En este documento vamos a explicar cómo usamos ghidra para decompilar un programa .exe hecho en C y como obtuvimos el código fuente de esta manera.

Preparativos

Para empezar debemos de hacer un programa en C y luego compilarlo usando TCC

```
ejemplo.c X
tcc-0.9.27-win32-bin > tcc > C ejemplo.c
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  int main() {
5      FILE *archivo = fopen("resultado.json", "w");
6
7      if (archivo == NULL) {
8          printf("Error al crear el archivo JSON.\n");
9          return 1;
10     }
11
12     fprintf(archivo, "{\n");
13     fprintf(archivo, "  \"equipo\": \"Equipo Verde\",\n");
14     fprintf(archivo, "  \"nom-integrantes\": \"mafer, mich, anibal\",\n");
15     fprintf(archivo, "  \"estado\": \"Exitoso\"\n}");
16     fprintf(archivo, "\n");
17
18     fclose(archivo);
19
20     printf("¡Archivo JSON generado con éxito!\n");
21     return 0;
22 }
```

Este código creara un archivo JSON en el que luego escribe información y manda un mensaje de confirmación si lo logra crear y escribir la información y uno de error si no lo logra

Usando tcc lo compilamos

```
PS C:\Users\thego\OneDrive\Escritorio\9no semestre\topico\code\tcc-0.9.27-win32-bin\tcc> .\tcc.exe .\ejemplo.c -o .\ejemplo.exe
PS C:\Users\thego\OneDrive\Escritorio\9no semestre\topico\code\tcc-0.9.27-win32-bin\tcc> |
```

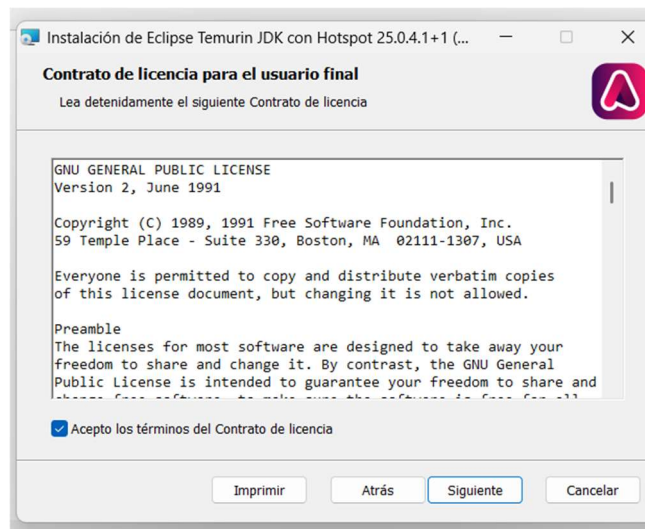
Al usar el exe desde la terminal vemos el mensaje de que el JSON se creo con éxito y el JSON con la información que le dimos en el código

```
PS C:\Users\thego\OneDrive\Escritorio\9no semestre\topico\code\tcc-0.9.27-win32-bin\tcc> .\ejemplo.exe
¡¡Archivo JSON generado con éxito!
```

```
{} resultado 18/09/2026 02:09 p. m. Archivo de origen ... 1 KB

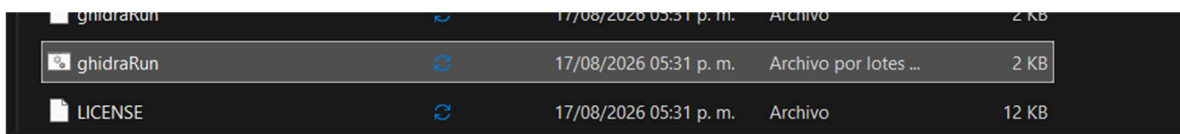
{} resultado.json X ejemplo.c X
tcc-0.9.27-win32-bin > tcc > {} resultado.json > ...
1 {
2   "equipo": "Equipo Verde",
3   "nom-integrantes": "mafer, mich, anibal",
4   "estado": "Exitoso"
5 }
6
```

Antes de empezar a usar Ghidra debemos instalar el jdk 25.0 de Eclipse que nos permitirá usar Ghidra

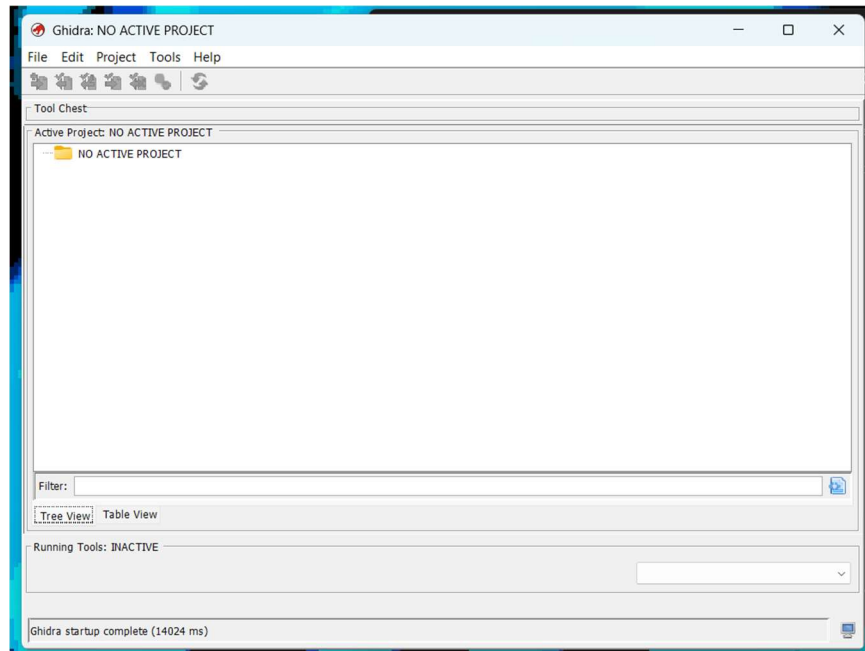


Uso de Ghidra para decompilar

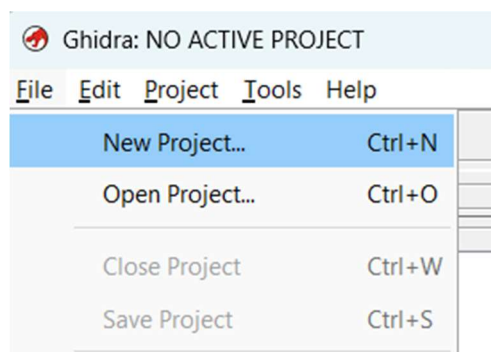
Una vez que tengamos el jdk instalado y descargamos el ghidra desde su pagina de github buscamos el archivo ghidrarun.bat y lo ejecutamos



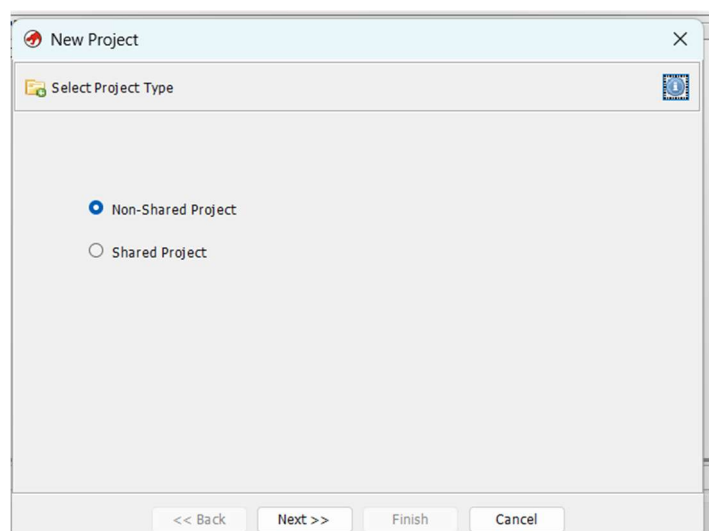
Esto nos abrirá ghidra en su pantalla principal, veremos que debemos de crear un proyecto para poder empezar a usarlo



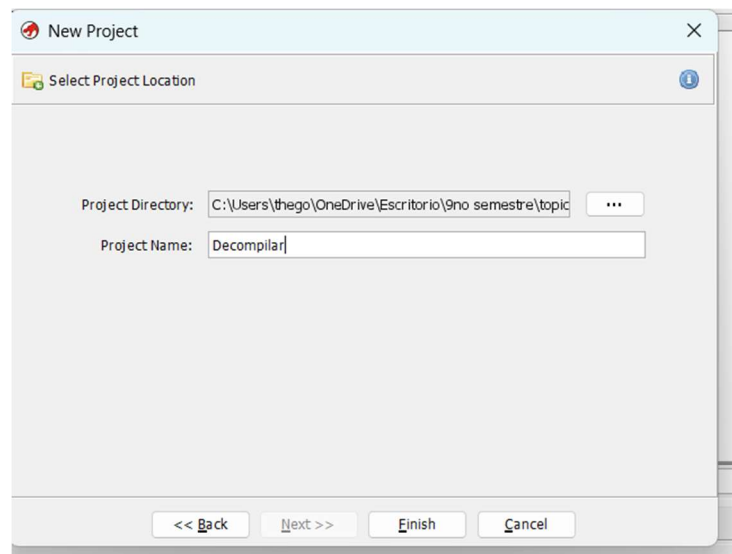
Para crear un nuevo proyecto nos vamos a File > New Project



Configuramos el tipo de proyecto como Non-Shared



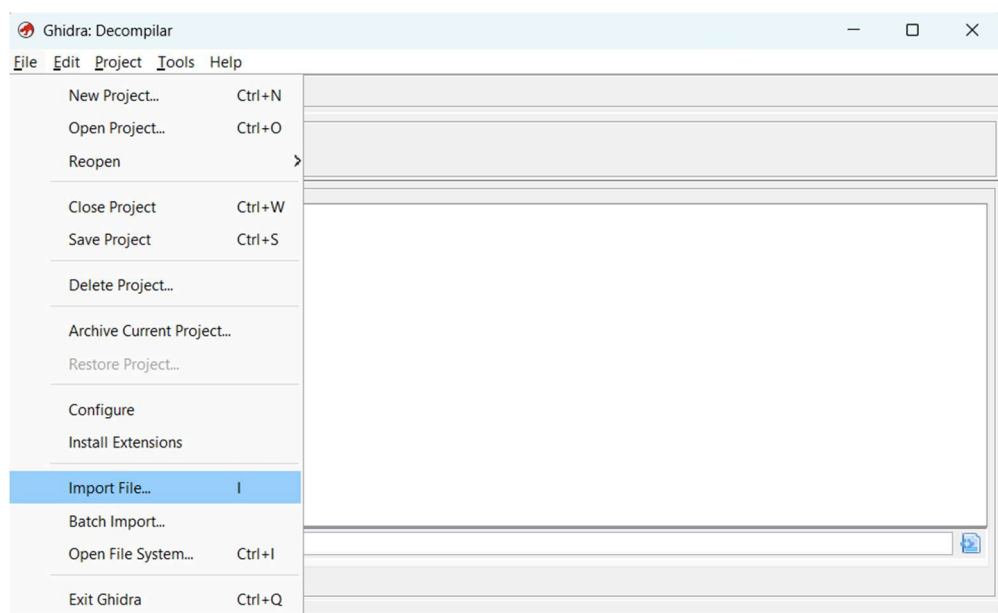
Elegimos el lugar donde lo vamos a guardar y el nombre del proyecto



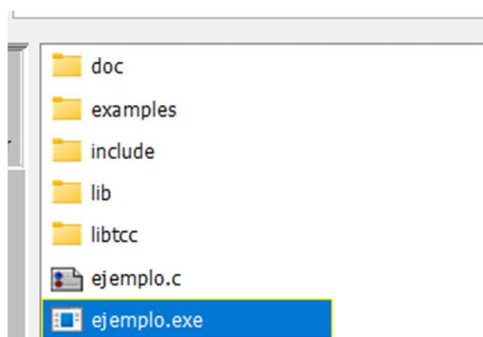
Tendremos un proyecto listo en la sección de proyectos activos



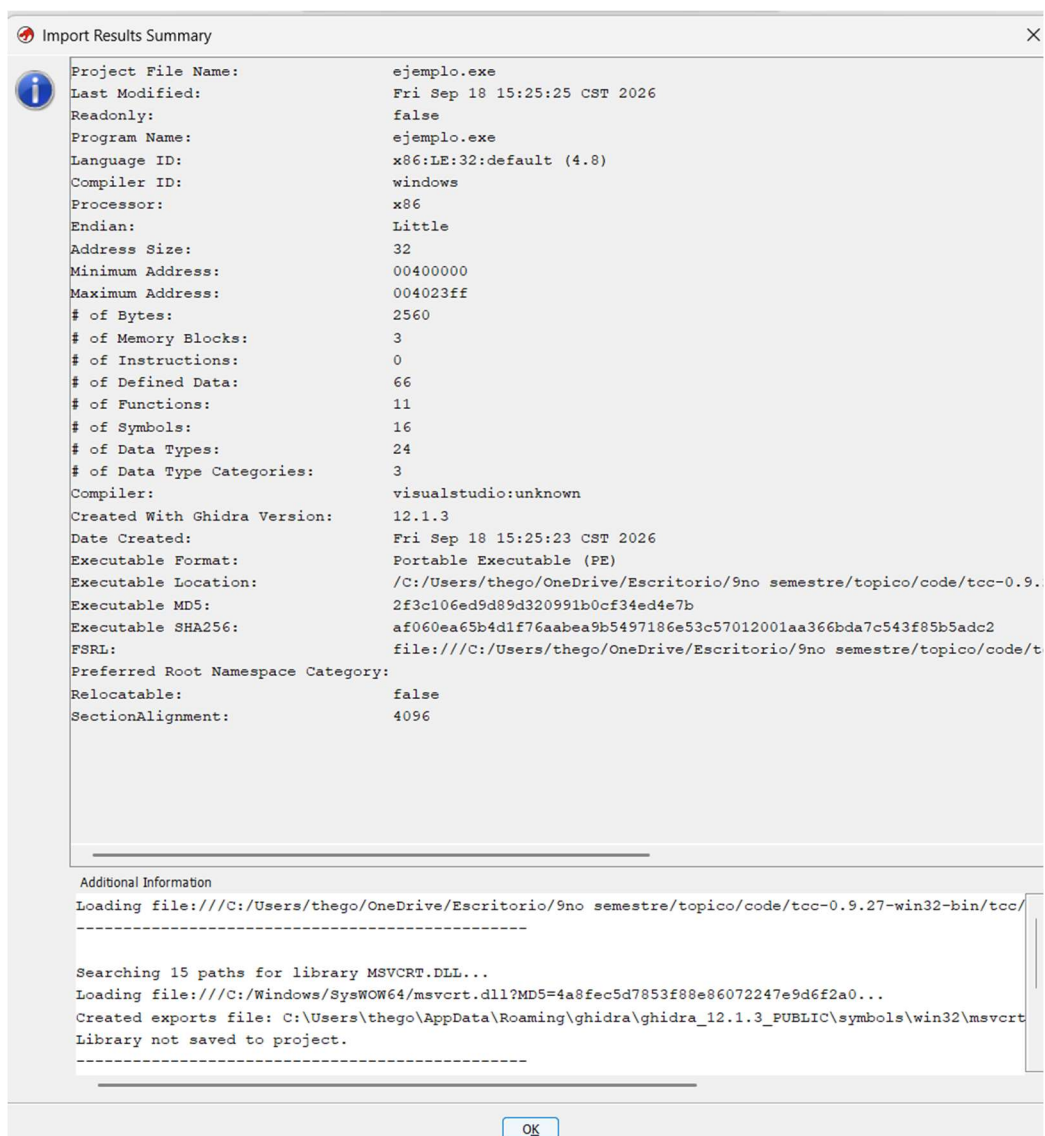
Para importar el ejecutable que hicimos previamente en c y compilamos como .exe nos vamos a ir a File>Import File



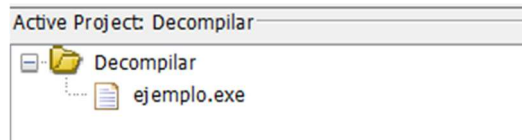
Seleccionamos el ejecutable de nuestro programa



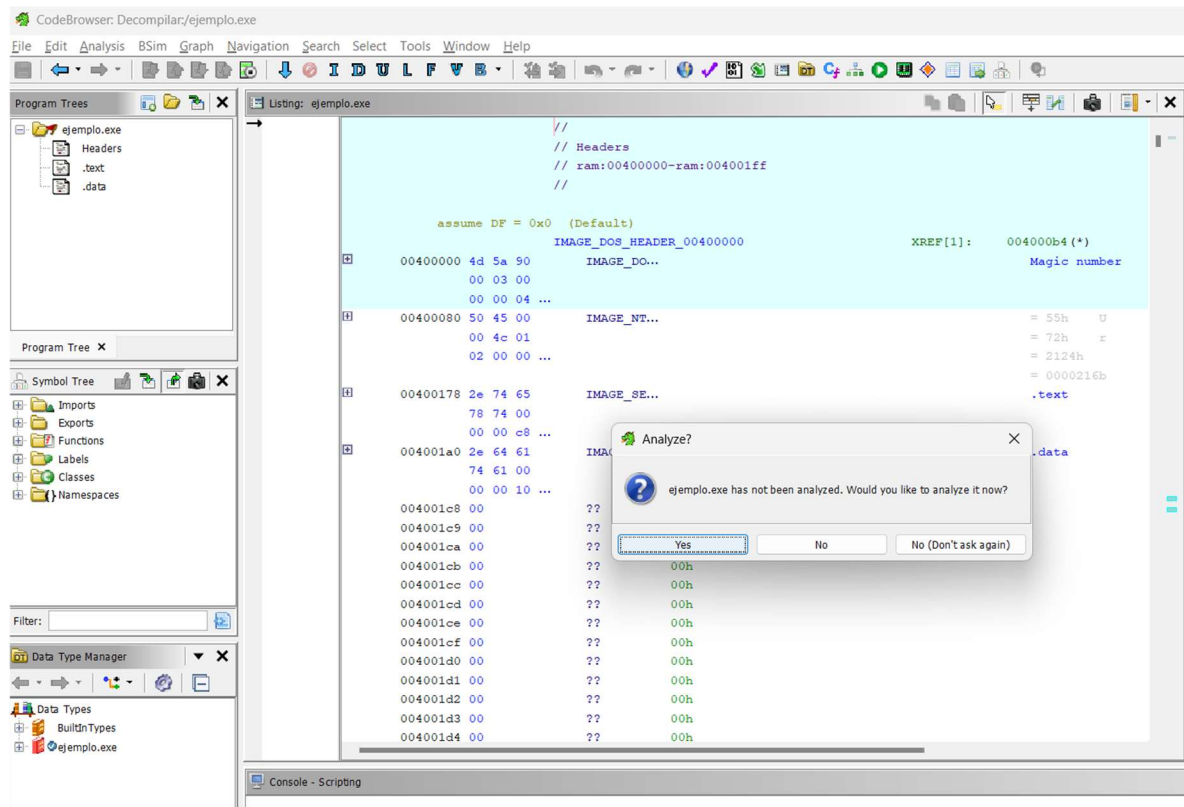
Nos da el siguiente resumen de datos de importación

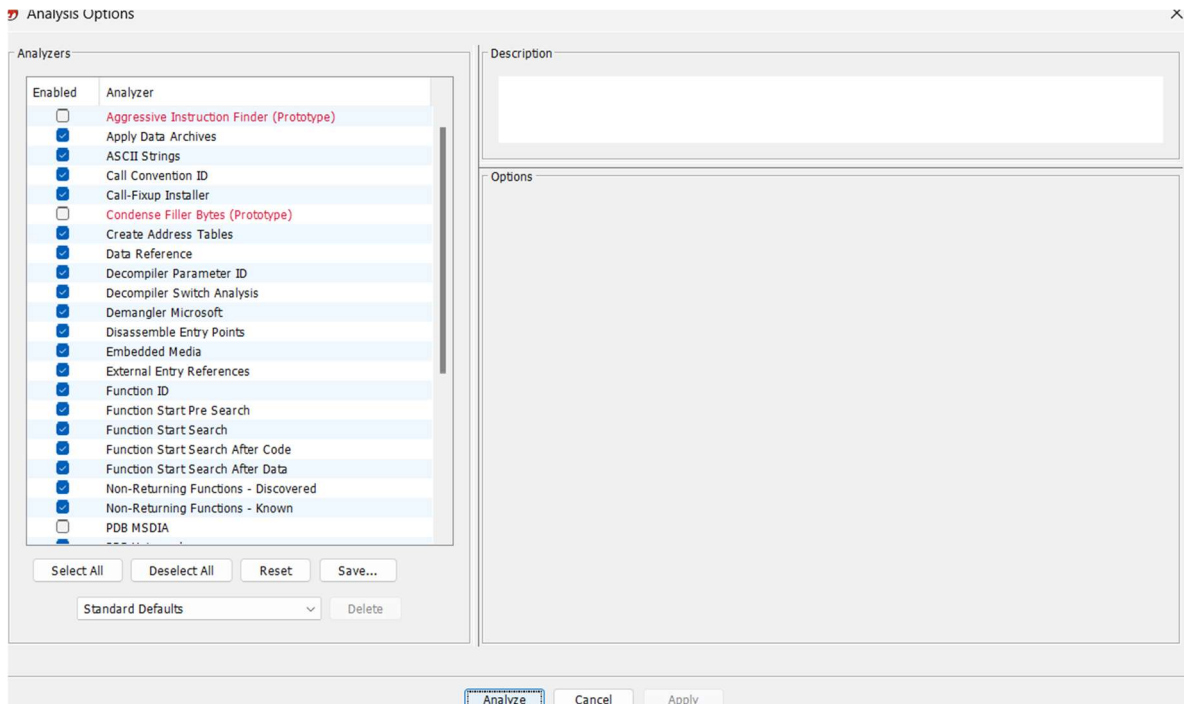


El ejecutable saldrá en nuestro panel de ghidra en la sección de proyectos activos, le daremos doble click lo que nos abrirá el CodeBrowser

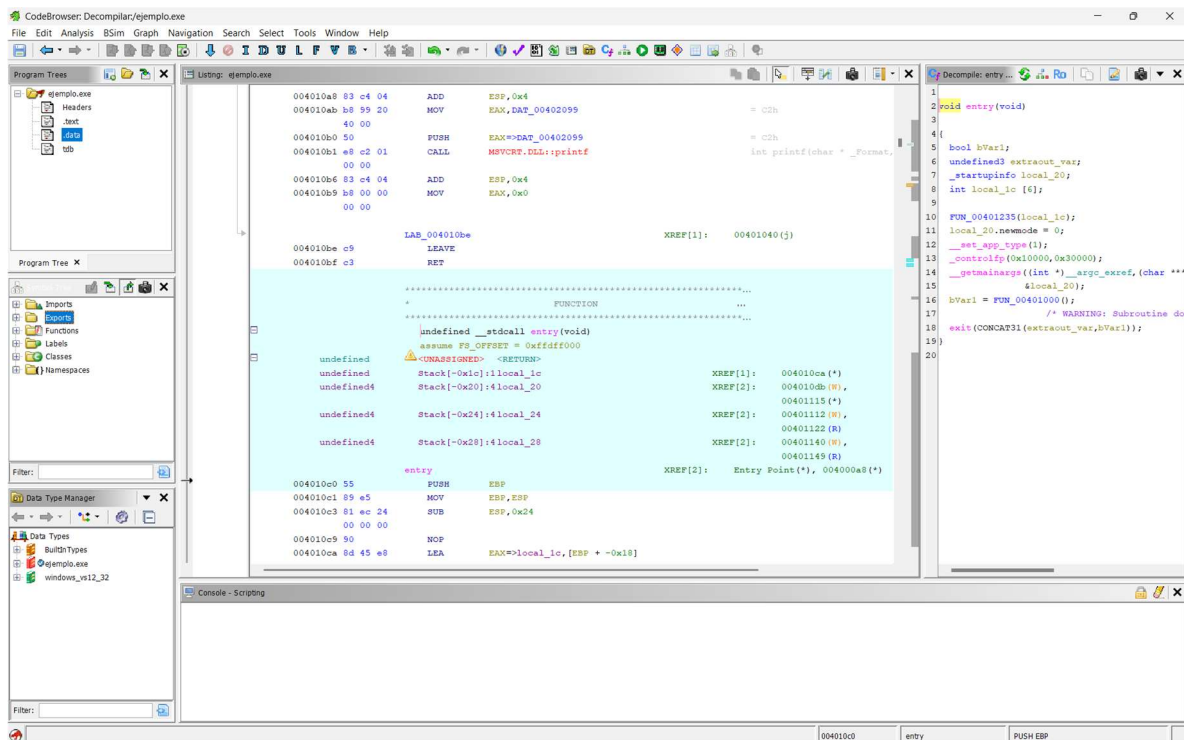


En el CodeBrowser le vamos a confirmar que analice el archivo

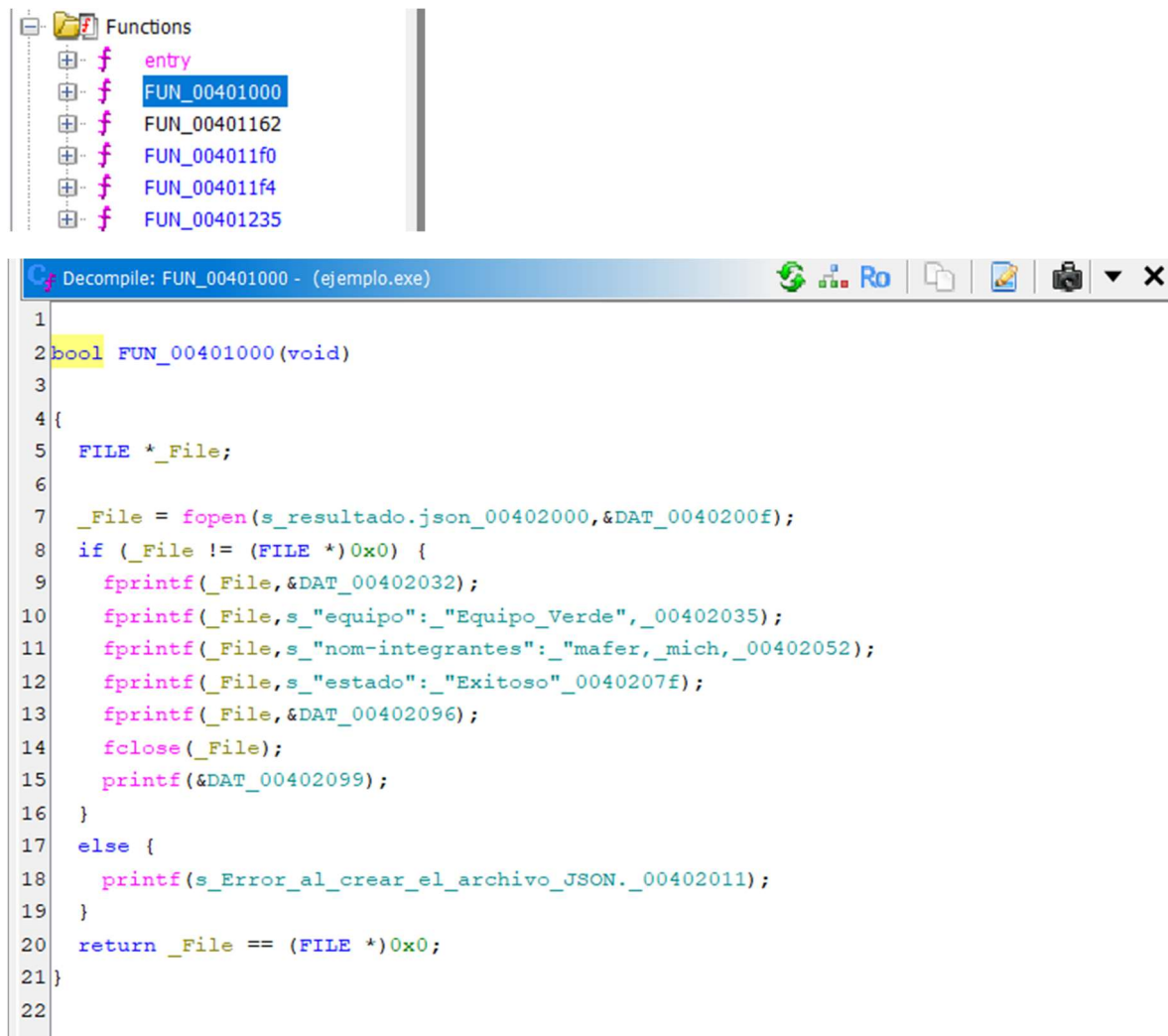




Nos dará el código del ejecutable, debemos de buscar el código que pertenece al main que hicimos



Buscamos en la sección Program tree que esta a la izquierda, en esta nos dirigimos a la sección functions, le damos click a cada una hasta que en la ventana de decompile a la derecha lo encontremos.



Este es el código decompilado del programa, lo vamos a limpiar para que pueda ser ejecutado de nuevo como código de c

Interpretación y reescritura del código

Teniendo el código decompilado por ghidra revisamos las sentencias y sus funciones, estas las documentamos en la siguiente tabla.

Codigo	Descripcion
bool FUN_00401000(void)	Esta función regresa un valor booleano, esta función corresponde a un int main()

FILE *_File;	Declara un apuntador tipo FILE * para poder crear y escribir archivos
_File = fopen(...)	Esta sentencia abre o crea un archivo llamado resultado.json
if (_File != (FILE *)0x0)	Esta sentencia verifica que el archivo se haya abierto exitosamente si el valor es diferente a NULL entonces el archivo fue creado exitosamente
fprintf	Estas sentencias se encargan de escribir la información que les demos en el archivo, en nuestro caso la estructura del json con informacion
fclose(_File);	Cierra el archivo que se edito
printf(...)	Esta sentencia escribe un mensaje en la consola
else { ... }	Corresponde a la parte que el código ejecutara en caso de que la condición no se haya cumplido (que el archivo no se haya podido abrir)
return ...	Esta sentencia devuelve el estado final de la ejecución, en C si el programa retorna un 0 significa que fue ejecutado exitosamente

Con esta información reescribimos el código manteniendo la estructura que nos dio ghidra, el código quedo así

```

C ejemplorecomp.c X
tcc-0.9.27-win32-bin > tcc > C ejemplorecomp.c
1
2  #include <stdio.h>
3  #include <stdlib.h>
4
5  int main() {
6      FILE *archivo = fopen("resultado.json", "w");
7
8      if (archivo != NULL) {
9          fprintf(archivo, "{\n");
10         fprintf(archivo, "  \"equipo\": \"Equipo_Verde\",\n");
11         fprintf(archivo, "  \"nom-integrantes\": \"mafer, mich\",\n");
12         fprintf(archivo, "  \"estado\": \"Exitoso\"\n");
13         fprintf(archivo, "}\n");
14
15         fclose(archivo);
16
17         printf("¡Archivo JSON generado con exito!\n");
18         return 0;
19     } else {
20         printf("Error al crear el archivo JSON.\n");
21         return 1;
22     }
23 }

```

Este código luego puede ser compilado de nuevo y debería funcionar igual que el original